

Java Exception

Chapter1

Java Exception 소개

Exception

- 코드에 문제가 발생할 때
현재 진행되고 있는 코드를
중단하고, 특별한 처리를 하는 것

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {

        //ArithmeticException
        int a = 3 / 0;

        //ArrayIndexOutOfBoundsException
        int[] arr = { 1, 2 };
        arr[5] = 1;

        //IndexOutOfBoundsException
        ArrayList<Integer> arr = new ArrayList<>();
        arr.get(0);
    }
}
```

Exception 이 발생하는 코드

Exception Handler

- throw 로 exception을 발생시킨다.
- Exception 발생 후, 어떻게 처리할 지 코드가 필요하다.
이를 Exception Handler 라고 한다.

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            throw new Exception("OH MY GOD");  
        } catch (Exception e) {  
        }  
    }  
}
```

Exception Handler가 없는 코드

- Throw를 포함한 함수에서, Exception에 대한 처리를 main 밖에서 처리 될 것임을 의미.
- 아래 코드에서는 Exception Handler가 없다.

```
public class Main {  
    public static void main(String[] args) throws Exception {  
        throw new Exception("OH MY GOD");  
    }  
}
```

발생된 Exception이
main 밖에서 처리 될 것임을 의미.

아래 코드의 문제점은?

- Exception Handler도 존재하지만 문법 에러이다.
- 어떤 문제일까?

```
public class Main {  
  
    public static void go() {  
        throw new Exception("OH MY GOD");  
    }  
  
    public static void main(String[] args) {  
        try {  
            go();  
        }  
        catch (Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

코드 문제 해결하기

- throws Exception 을

메서드 시그니처에 붙여 줘야함.

- Exception 클래스를 throw하는 함수에서는 직접 에러를 처리할지, 뒤로 미룰지 선택해야한다.

```
public class Main {  
  
    public static void go() throws Exception {  
        throw new Exception("OH MY GOD");  
    }  
  
    public static void main(String[] args) {  
        try {  
            go();  
        }  
        catch (Exception e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

선택적 Exception 처리하기

- 특정 Exception 발생시 처리되는 Handler 여러 개 삽입 가능

```
public class Main {  
    public static void go() {  
        int[] arr = {1, 2};  
        arr[5] = 10;  
    }  
  
    public static void main(String[] args) {  
        try {  
            go();  
        }  
        catch (Exception e) {  
            System.out.println("Exception!");  
        }  
    }  
}
```

Exception!

```
public class Main {  
    public static void go() {  
        //int a = 20 / 0;  
        int[] arr = {1, 2};  
        arr[5] = 10;  
    }  
  
    public static void main(String[] args) {  
        try {  
            go();  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("배열 범위 Exception");  
        }  
        catch (Exception e) {  
            System.out.println("Exception!");  
        }  
    }  
}
```

배열 범위 Exception

finally

- catch에서
한번 더 Exception 발생시
무조건 수행이 되게끔
하는 문법
- 예외 발생해도
반드시 정리작업을 해야 하는
경우 사용한다.
File Close 같은 코드가
finally 에 들어갈 수 있다.

```
public static void main(String[] args) {  
    try {  
        go();  
    }  
    catch (ArrayIndexOutOfBoundsException e) {  
        System.out.println("배열 범위 Exception");  
        int b = 20 / 0;  
    }  
    catch (Exception e) {  
        System.out.println("Exception!");  
    }  
    finally {  
        System.out.println("어쨌든 수행완료");  
    }  
}
```

배열 범위 Exception

어쨌든 수행완료

Exception in thread "main" java.lang.ArithmeticException: / by zero
at Main.main(Main.java:15)

Unchecked Exception (Runtime Exception)

- Exception 종류 중 하나.
- Exception 처럼 반드시 Handler가 있어야 하는 것이 아니다.
- Handler 코드를 넣어도 되고, 안 넣어도 된다.

```
public class Main {  
    public static void go() {  
        throw new RuntimeException("OH NO");  
    }  
  
    public static void main(String[] args) {  
        go();  
    }  
}
```

```
Exception in thread "main" java.lang.RuntimeException Create breakpoint : OH NO  
    at Main.go(Main.java:4)  
    at Main.main(Main.java:8)
```

Custom Exception 만들기

- 표준 Exception이 아닌,

더 명확한
Exception을 표현하고 싶을 때,

Custom Exception을 사용한다.

```
class KFCException extends RuntimeException {  
}  
  
public class Main {  
    public static void go() {  
        throw new KFCException();  
    }  
  
    public static void main(String[] args) {  
        go();  
    }  
}
```

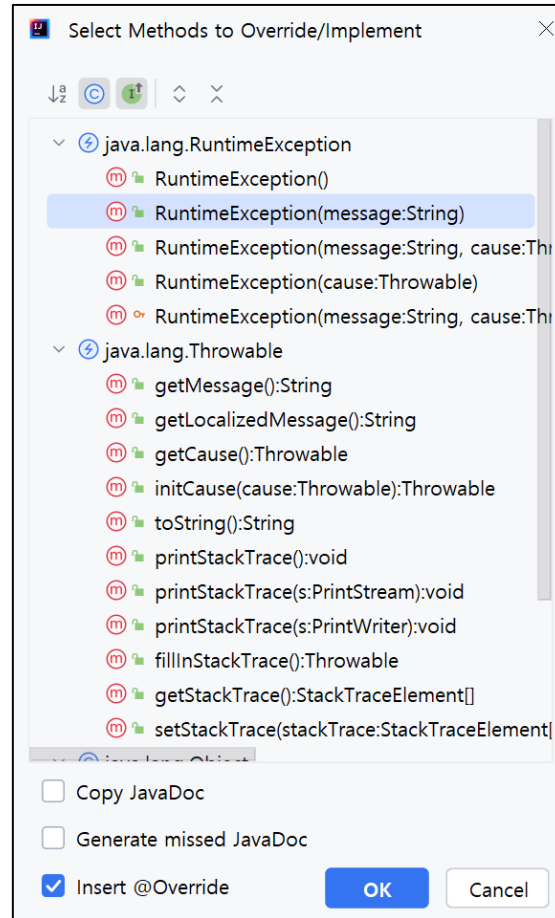
```
Exception in thread "main" KFCException  
    at Main.go(Main.java:6)  
    at Main.main(Main.java:10)
```

메시지를 포함하는 Custom Exception

```
class KFCException extends RuntimeException {  
}  
  
public void go() {  
    KFCException();  
}  
  
public static void main(String[] args) {  
    go();  
}
```

Generate

- Constructor
- toString()
- Override Methods...
- Delegate Methods...
- Test...
- Copyright



```
class KFCException extends RuntimeException {  
    public KFCException(String message) {  
        super(message);  
    }  
}  
  
public class Main {  
    public static void go() {  
        throw new KFCException("KFC가 아닙니다.");  
    }  
  
    public static void main(String[] args) {  
        go();  
    }  
}
```

Exception in thread "main" KFCException: KFC가 아닙니다.
at Main.go(Main.java:9)
at Main.main(Main.java:13)